

Basketball Team Management

Product Requirements Documentation

SAIGON ROVERS BASKETBALL CLUB

Product Requirements Documentation

This folder contains the product requirements documentation for **Team Management**.

How to read

- Start here: **Master PRD** → [00-master-prd](#)
- Auth & roles → [01-roles-permissions](#)
- Page-by-page specs → [pages/README](#)

Specs are organized **by page**, mirroring the app's sidebar navigation. Each page has one spec; its functionalities are sections inside that spec.

How to export a single PDF report

A GitHub Actions workflow stitches all the docs above into one PDF via md-to-pdf (Puppeteer/Chromium).

- **Workflow:** `.github/workflows/prd-pdf.yml` (named **PRD PDF**)
- **Triggers:**
 - Manually — Actions → **PRD PDF** → **Run workflow** (`workflow_dispatch`).
 - Automatically — on push to `main` that touches `docs/prd/**`.
- **Output:** the run uploads `sgr-team-management-prd.pdf` as a build artifact. Open the workflow run → **Artifacts** → download **Product Requirements Document**.

To preview locally (requires a one-time `npm install md-to-pdf`):

```
node scripts/build-prd-pdf.mjs
```

→ produces `sgr-team-management-prd.pdf` in the repo root. Section order is defined by the `MANIFEST` array in that script (add a page there); styling lives in `scripts/prd-pdf.css`.

Master PRD — Saigon Rovers Basketball Team Management

1. Product Overview

Team Management is a web app for managing teams, people, and operations (assets, schedules, etc.) with role-based access.

Problem

Teams often track key operational info in spreadsheets/chats, causing:

- stale information
- unclear ownership
- poor visibility across roles

Solution

Provide a centralized, role-based hub where users can view and (when authorized) manage team data and workflows.

- Application is a private team portal
- Users are trusted team members (invited via email by admin)
- The role set is small and static

2. Target Users

- **GUEST:** limited read-only access / Different database with mock data?
- **PLAYER:** participant
- **COACH:** coaching staff
- **SUPER_ADMIN:** global/admin operations
- **Captain (PLAYER flag):** elevated permissions within a team

3. Primary Use Cases (Jobs-to-be-done)

- A player checks team info and performance metrics.
- A coach reviews team status and metrics.
- A captain manages operational items like **Assets**.
- An admin manages teams and configuration safely.

4. Feature Map (High-level)

This is a high-level map; details live in the [Page Specs](#) docs.

5. Product Principles (Shared UX expectations)

- **Role clarity:** users should never wonder why they can/can't do something.
- **Fast search:** lists must support filtering.
- **Safe destructive actions:** deletions require confirmation + clear feedback.
- **Deep-linkable state:** filters should be reflected in the URL where reasonable.

6. Shared Requirements (Cross-feature)

Authorization

- UI may hide controls, but **server must enforce** permissions.

Audit / Logging (optional)

- ☐ For privileged actions (create/edit/delete), capture: actor role, team context, timestamp.

7. Non-Functional Requirements

- **Performance:** common pages load quickly for typical team sizes.
- **Accessibility:** keyboard navigation and accessible dialogs.
- **Reliability:** clear loading/error states; avoid duplicate submits.

8. Release / Rollout (Light)

- Prefer feature flags for larger changes.
- Ship in small increments with backward-compatible data changes.

9. Open Questions

- What are the most important product-level success metrics?
- Which features require audit logs vs. basic activity logging?

Authentication & Authorization

Layer 1: Proxy

```
proxy.ts
```

Route guard for page navigation only.

→ Block unauthenticated/ expired page navigation and RBAC before hitting the route handler.

Layer 2: withAuth / withResource

```
auth.api.getSession
```

- Only layer that cannot be bypassed.
- Always runs for every server action.
- Provide `user` context.

Layer 3: Layout

```
authClient.useSession
```

Proactive client-side UX

Roles, Permissions & Glossary

1. Roles (Definitions)

GUEST

- Read-only access to view data.

PLAYER

- Standard team member access.

COACH

- Staff-level access (typically view + coaching operations).

SUPER_ADMIN

- Administrative access across the application.

Captain (PLAYER flag)

A PLAYER with elevated permissions for specific team-level operations.

2. Permission Model (General rules)

2.1. Server side

Actions are enforced with `withResource` → defense-in-depth

2.2. Client (UI) side

Hide actions or UI elements with `usePermissions` :

- `isAdmin`, `isCoach`, `isPlayer`, `isGuest`
- `can`
- `canAll`
- `canAny`

The rule of thumb: if you're hiding an action button (create/edit/delete), use `Authorized` . Everything else stays `Visibility` .

2.3. Proxy

- Items on sidebar are hidden by permissions.
- Direct URL / API access will be rejected when unauthorized or don't have permissions.

3. Glossary

- **RBAC**: Role-Based Access Control
- **Query params**: URL parameters like `?q=ball&condition=Good` used to store filter state.

Page Specs

Docs are organized **by page**, mirroring the app's sidebar navigation (`SIDEBAR_GROUP` in `src/app/(protected)/_helpers/utils.ts`). Each page has one spec; its individual functionalities are sections **inside** that spec (§5 Functional Requirements).

Each spec follows the same template:

- User-facing behavior (readable by end users)
- Requirements + Acceptance Criteria (for engineering/QA)
- Technical appendix (data/API/permissions) if possible.

Page catalog

Nav group	Page	Route	Status	Spec
Overview	Dashboard	<code>/dashboard</code>	Draft	dashboard
Overview	Team Rule	<code>/team-rule</code>	Draft TODO	team-rule
Team Management	Roster	<code>/roster</code>	Draft TODO	roster
Team Management	Training	<code>/training</code>	Draft TODO	training
Team Management	Attendance	<code>/attendance</code>	Draft TODO	attendance
Team Management	Registration	<code>/registration</code>	Draft TODO	registration
Team Management	Matches	<code>/matches</code>	Draft TODO	matches
Performance	Periodic Testing	<code>/periodic-testing</code>	Draft TODO	periodic-testing
Resources	Assets	<code>/assets</code>	Draft	assets
Settings	Leagues	<code>/leagues</code>	Draft TODO	leagues
Settings	Locations	<code>/locations</code>	Draft TODO	locations

Dashboard includes the **PDF Reports** capability (formerly the standalone "Analytics Reports" spec) as a functionality under §5.

Not yet documented

- Auth pages: `/login` , `/forgot-password` , `/new-password`
- User profile: `/profile/[id]`
- Resources → **Documents** (`/documents` , disabled in the sidebar)

Dashboard

Route: /dashboard · Nav group: **Overview** · Status: **Draft**

1. Summary

- The **Dashboard** is the default landing page after login (`DEFAULT_LOGIN_REDIRECT`).
- It presents team analytics (charts, key figures) for a selected time `interval` .
- A **PDF Reports** capability lets users export the analytics exactly as rendered on screen (see §5 — *PDF Reports*).

2. Goals / Metrics

Goals

- Give every role an at-a-glance view of team health and activity.
- Let authorized users export/share the analytics as a print-ready PDF.

Metrics

- Time to first meaningful render of the dashboard.
- Time to produce a PDF report (render + serve).

3. Users & Permissions

Role	View dashboard	Manual PDF download	Configure schedules	View reports history
GUEST	Yes	No	No	No
PLAYER	Yes	Yes	No	No
COACH	Yes	Yes	No	Yes
SUPER_ADMIN	Yes	Yes	Yes	Yes
PLAYER (Captain)	Yes	Yes	Yes	Yes

PDF-related permissions are a starting proposal — confirm before building the scheduling/history phases.

4. UX / Flows

Entry point

- Sidebar → **Dashboard** (also the post-login redirect).

View

- Analytics are scoped by an `interval` selector carried in the URL query params.

Export (PDF Reports)

- A "Download report" action posts the current dashboard URL to the report API and streams back a PDF that matches the on-screen analytics for the selected interval.

5. Functional Requirements

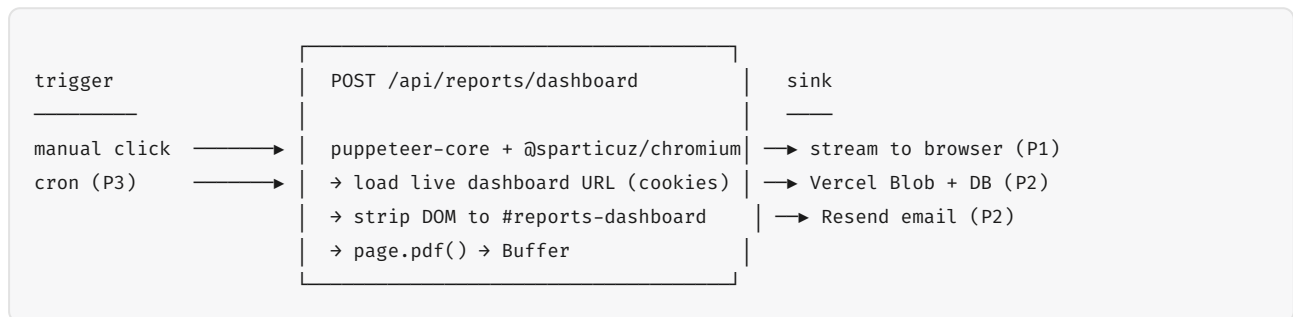
Analytics view

- **FR-1:** All roles can view the dashboard analytics.
- **FR-2:** The `interval` selector updates the analytics and is reflected in the URL.

PDF Reports

Folded in from the former "Analytics Reports" feature spec. The rendering engine is built **once** and reused across all phases; only the *sink* (where bytes go) and the *trigger* (who starts it) change.

Architecture overview



- **Why Puppeteer (not pdf-lib):** the dashboard uses Recharts (client-side SVG). High fidelity requires a real browser render, which `pdf-lib` cannot do.
- **Why reuse the live dashboard (no dedicated print route):** the client posts its current URL; the server forwards the session cookies, renders that page, then strips the DOM down to the `#reports-dashboard` grid (dropping nav and filters) and forces a 2-charts-per-row layout. This avoids maintaining a parallel print page — the report stays in sync with the dashboard for free.
- **Deployment:** Vercel/serverless ⇒ `puppeteer-core + @sparticuz/chromium` (bundled Puppeteer's ~300MB Chromium won't fit). Set route `maxDuration` high enough for Puppeteer cold start + render.

Phase 1 — Manual download

- **FR-3:** A "Download report" action on the Dashboard posts the current URL to `POST /api/reports/dashboard`, producing a PDF that reflects the selected `interval` (carried in the URL's query params).
- **FR-4:** The PDF visually matches the on-screen dashboard analytics: the server renders the live page, keeps only the `#reports-dashboard` grid, and lays the charts out 2 per row.
- **FR-5:** Generation runs server-side in a single route handler; nothing is persisted and no file is written to disk (bytes stream straight to the client).
- **FR-6:** The client shows a loading state during generation and a toast on error.
- **FR-7:** Non-ASCII (Vietnamese) filenames are preserved (RFC 5987 `filename*`).

Phase 2 — Persistence + email

- **FR-8:** Generated PDFs upload to Vercel Blob.
- **FR-9:** Each report writes a metadata row (see Technical Appendix).
- **FR-10:** Downloads are served through an auth-guarded route — raw Blob URLs are never exposed (player data is sensitive).
- **FR-11:** A report can be emailed to a recipient list via Resend (attachment or link to the download route).

Phase 3 — Scheduling + reports page

- **FR-12:** Users can configure schedules (frequency, recipients, time range).
- **FR-13:** A cron trigger generates due reports using the same engine.
- **FR-14:** A reports page lists history with status, period, and download link.
- **FR-15:** Failed cron runs are recorded with an error and surfaced in the list.
- **FR-16:** A retention policy deletes old Blobs and marks rows `expired`.

6. Acceptance Criteria (Given/When/Then)

- **AC-1:** Given I am SUPER_ADMIN on the Dashboard with `interval=last_30_days`, when I click "Download report", then I receive a PDF matching the on-screen analytics for that interval.
- **AC-2:** Given generation fails, when I click download, then I see an error toast and no file is downloaded.
- **AC-3 (P2):** Given a report is generated by cron, then a metadata row exists with `trigger=cron`, a `blob_pathname`, and `status=success`.
- **AC-4 (P2):** Given a report exists, when I open its download link, then access is rejected without a valid session.
- **AC-5 (P3):** Given a weekly schedule, when its time arrives, then a report is generated, stored, and emailed to the configured recipients.

7. Technical Appendix

Shared report engine (Phase 1)

- `POST /api/reports/dashboard` — accepts `{ url }`, launches Chromium, forwards the request's session cookies, renders the live dashboard, strips the DOM to `#reports-dashboard` (2 columns), and returns PDF bytes via `page.pdf()` as a single continuous page.
- Browser is always closed in a `finally` block; no temp file is written.
- Download trigger helpers (`triggerDownload`, `withExtension`) extracted from `registration/_helpers/pdf.ts` into a shared `lib/download.ts`.

Data model — `reports` (Phase 2, logical)

- `id` : `uuid`
- `created_at` : `timestamp`
- `period_start` / `period_end` : `timestamp` (what the report covers)
- `trigger` : `enum [ manual-archived , cron ]`
- `schedule_id` : `uuid` (FK → `report_schedules`, nullable)
- `blob_pathname` : `string`
- `file_size` : `integer`
- `status` : `enum [ success , failed , expired ]`
- `error` : `string` (nullable)
- `emailed_to` : `string[]` (nullable)
- `emailed_at` : `timestamp` (nullable)

Data model — `report_schedules` (Phase 3, logical)

- `id` : `uuid`
- `frequency` : `enum [ daily , weekly , monthly ]`
- `recipients` : `string[]`
- `interval` : `string` (analytics time range the report covers)
- `last_run_at` : `timestamp` (nullable)
- `enabled` : `boolean`

Dependencies to add

- `puppeteer-core`, `@sparticuz/chromium` (Phase 1)
- `@vercel/blob` (Phase 2)
- Reuse: `resend` (already configured in `src/lib/auth.ts`)

Open questions

- Recipients: fixed admin list or per-user/per-team configurable?
- Retention window (e.g. 6 months)?
- Should manual downloads ever opt into being archived?

Team Rule

Route: `/team-rule` · Nav group: **Overview** · Status: **Draft** — **TODO**

1. Summary

- *TODO*: describe what the Team Rule page shows and why it exists.

2. Goals / Metrics

- *TODO*

3. Users & Permissions

Role	View	Edit
GUEST	TODO	TODO
PLAYER	TODO	TODO
COACH	TODO	TODO
SUPER_ADMIN	TODO	TODO
PLAYER (Captain)	TODO	TODO

Confirm against resource: `'team-rule'` permissions in `src/app/(protected)/_helpers/utils.ts` and the permission model.

4. UX / Flows

- Entry point: Sidebar → **Team Rule**.
- *TODO*: view / edit flows.

5. Functional Requirements

- *TODO*: list each functionality as a subsection (FR-1, FR-2, ...).

6. Acceptance Criteria (Given/When/Then)

- *TODO*

7. Technical Appendix

- *TODO*: data model, API, query params.

Roster

Route: `/roster` · Nav group: **Team Management** · Status: **Draft** — **TODO**

1. Summary

- TODO*: describe the team roster page (members list, profiles, etc.).

2. Goals / Metrics

- TODO*

3. Users & Permissions

Role	View	Add	Edit	Delete
GUEST	TODO	TODO	TODO	TODO
PLAYER	TODO	TODO	TODO	TODO
COACH	TODO	TODO	TODO	TODO
SUPER_ADMIN	TODO	TODO	TODO	TODO
PLAYER (Captain)	TODO	TODO	TODO	TODO

Confirm against `resource: 'roster'` permissions in `src/app/(protected)/_helpers/utils.ts`. Note: individual member profiles live at `/profile/[id]`.

4. UX / Flows

- Entry point: Sidebar → **Roster**.
- TODO*

5. Functional Requirements

- TODO*

6. Acceptance Criteria (Given/When/Then)

- TODO*

7. Technical Appendix

- TODO*

Training

Route: `/training` · Nav group: **Team Management** · Status: **Draft** — **TODO**

1. Summary

- TODO*: describe the training sessions page.

2. Goals / Metrics

- TODO*

3. Users & Permissions

Role	View	Add	Edit	Delete
GUEST	TODO	TODO	TODO	TODO
PLAYER	TODO	TODO	TODO	TODO
COACH	TODO	TODO	TODO	TODO
SUPER_ADMIN	TODO	TODO	TODO	TODO
PLAYER (Captain)	TODO	TODO	TODO	TODO

Confirm against `resource: 'training'` permissions in `src/app/(protected)/_helpers/utils.ts`.

4. UX / Flows

- Entry point: Sidebar → **Training**.
- TODO*

5. Functional Requirements

- TODO*

6. Acceptance Criteria (Given/When/Then)

- TODO*

7. Technical Appendix

- TODO*

Attendance

Route: /attendance · Nav group: **Team Management** · Status: **Draft — TODO**

1. Summary

- *TODO*: describe attendance tracking.

2. Goals / Metrics

- *TODO*

3. Users & Permissions

Role	View	Mark	Edit	Delete
GUEST	TODO	TODO	TODO	TODO
PLAYER	TODO	TODO	TODO	TODO
COACH	TODO	TODO	TODO	TODO
SUPER_ADMIN	TODO	TODO	TODO	TODO
PLAYER (Captain)	TODO	TODO	TODO	TODO

Confirm against resource: 'attendance' permissions in `src/app/(protected)/_helpers/utils.ts`.

4. UX / Flows

- Entry point: Sidebar → **Attendance**.
- *TODO*

5. Functional Requirements

- *TODO*

6. Acceptance Criteria (Given/When/Then)

- *TODO*

7. Technical Appendix

- *TODO*

Registration

Route: `/registration` · Nav group: **Team Management** · Status: **Draft — TODO**

1. Summary

- TODO*: describe player registration (and the registration PDF, if applicable — see `registration/_helpers/pdf.ts`).

2. Goals / Metrics

- TODO*

3. Users & Permissions

Role	View	Register	Edit	Delete
GUEST	TODO	TODO	TODO	TODO
PLAYER	TODO	TODO	TODO	TODO
COACH	TODO	TODO	TODO	TODO
SUPER_ADMIN	TODO	TODO	TODO	TODO
PLAYER (Captain)	TODO	TODO	TODO	TODO

Confirm against resource: `'registration'` permissions in `src/app/(protected)/_helpers/utils.ts`.

4. UX / Flows

- Entry point: Sidebar → **Registration**.
- TODO*

5. Functional Requirements

- TODO*

6. Acceptance Criteria (Given/When/Then)

- TODO*

7. Technical Appendix

- TODO*

Matches

Route: `/matches` · Nav group: **Team Management** · Status: **Draft** — **TODO**

1. Summary

- TODO*: describe match management (schedule, results, etc.).

2. Goals / Metrics

- TODO*

3. Users & Permissions

Role	View	Add	Edit	Delete
GUEST	TODO	TODO	TODO	TODO
PLAYER	TODO	TODO	TODO	TODO
COACH	TODO	TODO	TODO	TODO
SUPER_ADMIN	TODO	TODO	TODO	TODO
PLAYER (Captain)	TODO	TODO	TODO	TODO

Confirm against resource: `'matches'` permissions in `src/app/(protected)/_helpers/utils.ts` . Related: `/leagues` , `/locations` .

4. UX / Flows

- Entry point: Sidebar → **Matches**.
- TODO*

5. Functional Requirements

- TODO*

6. Acceptance Criteria (Given/When/Then)

- TODO*

7. Technical Appendix

- TODO*

Periodic Testing

Route: `/periodic-testing` · Nav group: **Performance** · Status: **Draft** — **TODO** Sub-routes: `/periodic-testing/test-types`, `/periodic-testing/add-result`

1. Summary

- TODO*: describe performance/periodic testing — recording results over time.

2. Goals / Metrics

- TODO*

3. Users & Permissions

Role	View results	Add result	Manage test types	Edit	Delete
GUEST	TODO	TODO	TODO	TODO	TODO
PLAYER	TODO	TODO	TODO	TODO	TODO
COACH	TODO	TODO	TODO	TODO	TODO
SUPER_ADMIN	TODO	TODO	TODO	TODO	TODO
PLAYER (Captain)	TODO	TODO	TODO	TODO	TODO

Confirm against `resource: 'periodic-testing'` permissions in `src/app/(protected)/_helpers/utils.ts`.

4. UX / Flows

- Entry point: Sidebar → **Periodic Testing**.
- Add result** sub-flow → `/periodic-testing/add-result`.
- Test types** management sub-flow → `/periodic-testing/test-types`.
- TODO*: detail each sub-flow.

5. Functional Requirements

Results

- TODO*

Add result (`/add-result`)

- TODO*

Test types (`/test-types`)

- TODO*

6. Acceptance Criteria (Given/When/Then)

- TODO*

7. Technical Appendix

- *TODO*

Assets

Route: /assets · Nav group: **Resources** · Status: **Draft**

1. Summary

- **Assets** page lets users view team assets and key metrics.
- Specific roles (SUPER_ADMIN and Captain) can add, edit, and delete assets.

2. Goals / Metrics

Goals

- Users can find assets quickly by name or condition.
- Captains/admins can keep inventory accurate with minimal effort.

Metrics

- The total number of assets.
- The number of assets that need to be replaced (condition = "Poor").

3. Users & Permissions

Role	View assets + metrics	Filter	Add	Edit	Bulk delete
GUEST	Yes	Yes	No	No	No
PLAYER	Yes	Yes	No*	No*	No*
COACH	Yes	Yes	No	No	No
SUPER_ADMIN	Yes	Yes	Yes	Yes	Yes
PLAYER (Captain)	Yes	Yes	Yes	Yes	Yes

*Captain is a subset of PLAYER.

4. UX / Flows

Entry point

- Sidebar → **Assets**

View & filter

- The Metrics section shows total assets and assets needing replacement. Clicking a metric card filters the list.
- List view shows assets and supports filtering by:
 - name (text)
 - category (select)
 - condition (select)
- Filter state is stored in URL query params.

Create

- Authorized users see **+ Add**
- Clicking **+ Add** opens a dialog.

Edit

- Non-guest users can click an asset row to open the same dialog with its data.

Delete

- Authorized users see a checkbox column.
- Selecting one or more assets enables deletion (popover).

5. Functional Requirements

Viewing

- **FR-1:** All roles can view the assets list.
- **FR-2:** All roles can view asset metrics section.

Filtering + query params

- **FR-3:** Filter by name, category, and condition.
- **FR-4:** URL reflects filters (query params).
- **FR-5:** Back/forward restores filters from the URL.

Create/edit

- **FR-6:** Only SUPER_ADMIN and Captain can create assets.
- **FR-7:** Only SUPER_ADMIN and Captain can edit assets.
- **FR-8:** Asset dialog fields:
 - Required: Name, Quantity
 - Optional: Category (default=Equipment), Condition (default=Good), Note
- **FR-9:** After a successful create or edit, the list and metrics update without a full page refresh.

Delete

- **FR-10:** Only SUPER_ADMIN and Captain can bulk delete assets.
- **FR-11:** Deletion shows success/error toast message.
- **FR-12:** After a successful deletion, the list and metrics update without a full page refresh.

6. Acceptance Criteria (Given/When/Then)

- **AC-1:** Given I am a GUEST, when I open Assets, then I do not see "+ Add" or delete checkboxes.
- **AC-2:** Given I am a Captain, when I add an asset with Name and Quantity, then it is created and visible in the list.
- **AC-3:** Given I filter by name "net", category "Equipment" and condition "Good", then the URL includes `q=net` , `category=EQUIPMENT` and `condition=GOOD` .
- **AC-4:** Given I am a COACH, when I attempt to delete assets (UI or API), then the request is rejected.

7. Technical Appendix

Data model (logical)

Asset:

- `name` : string (required)
- `quantity` : integer (default 1)
- `category` : enum [default "Equipment", "Training", "Others"]
- `condition` : enum [default "Good", "Fair", "Poor"]
- `note` : string (optional)

Query params contract

- `q` (string): name search
- `category` (string): category filter value
- `condition` (string): condition filter value
- `page` (number): current page

Example:

- `?q=ball&category=EQUIPMENT&condition=GOOD&page=1`

Leagues

Route: `/leagues` · Nav group: **Settings** · Status: **Draft** — **TODO**

1. Summary

- TODO*: describe league configuration (used by Matches).

2. Goals / Metrics

- TODO*

3. Users & Permissions

Role	View	Add	Edit	Delete
GUEST	TODO	TODO	TODO	TODO
PLAYER	TODO	TODO	TODO	TODO
COACH	TODO	TODO	TODO	TODO
SUPER_ADMIN	TODO	TODO	TODO	TODO
PLAYER (Captain)	TODO	TODO	TODO	TODO

Confirm against `resource: 'leagues'` permissions in `src/app/(protected)/_helpers/utils.ts`.

4. UX / Flows

- Entry point: Sidebar → **Leagues**.
- TODO*

5. Functional Requirements

- TODO*

6. Acceptance Criteria (Given/When/Then)

- TODO*

7. Technical Appendix

- TODO*

Locations

Route: `/locations` · Nav group: **Settings** · Status: **Draft** — **TODO**

1. Summary

- TODO*: describe venue/location configuration (used by Matches/Training).

2. Goals / Metrics

- TODO*

3. Users & Permissions

Role	View	Add	Edit	Delete
GUEST	TODO	TODO	TODO	TODO
PLAYER	TODO	TODO	TODO	TODO
COACH	TODO	TODO	TODO	TODO
SUPER_ADMIN	TODO	TODO	TODO	TODO
PLAYER (Captain)	TODO	TODO	TODO	TODO

Confirm against `resource: 'locations'` permissions in `src/app/(protected)/_helpers/utils.ts` .

4. UX / Flows

- Entry point: Sidebar → **Locations**.
- TODO*

5. Functional Requirements

- TODO*

6. Acceptance Criteria (Given/When/Then)

- TODO*

7. Technical Appendix

- TODO*